

Sprinkler Sense

이름: 정동승

학번: 2118208

Github:

https://github.com/dongseung0410/01_Python_and_Git/tree/main/%EB%8D%B0%EC%9D%B4%ED%84%B0%20%ED%94%84%EB%A0%88%EC%9E%84/%EA%B8%B0%EB%A7%90%EA%B3%A0%EC%82%AC

1. 제목 선정 이유

Sense 라는 단어를 통해 단순한 데이터 분석을 넘어, 데이터를 통해 감각적으로 인사이트를 도출하고 설계 최적화를 지원한다는 점을 강조하기 위해 Sprinkler Sense 로 머신 러닝 모델 이름을 정해보았다.

2. 안전 관련 머신 러닝 모델 개발 관련 요약

이 프로젝트는 소방 설계 및 설비 분야에서 활용할 수 있는 안전 관련 머신 러닝 모델을 개발하는 것을 목표로 한다. 특히, 건물 내 스프링클러 설계를 최적화하기 위해 머신 러닝 알고리즘을 적용하였다. 이 모델은 건물의 면적(Surface), 온도(Temperature), 기준 온도(BaseTemperature), 특정 거리(Distance), 요일별 공휴일 여부와 같은 독립 변수를 활용하여 스프링클러의 최적 배치 및 설계 방향을 제시한다.

모델 개발에 사용된 데이터는 **Kaggle** 의 **Buildings Energy Consumption Dataset** 에서 가져왔으며, 데이터가 너무 크고 분산되어 있어 1~6 까지의 파일을 하나로 합쳐 데이터를 활용하였다. 이를 통해 데이터의 일관성을 확보하고 분석의 정확도를 높였다.

모델은 스프링클러 설치 및 유지보수 과정에서 비용을 절감하고 자원의 효율적 관리를 실현하며, 필요 이상의 장비 설치를 방지하고 유지보수 비용을 줄일 수 있다. 또한, 스프링클러 작동에 영향을 미치는 주요 변수들을 분석하여 화재 발생 가능성을 예측하고, 이를 바탕으로 안전성을 극대화할 수 있는 설계 방안을 마련한다. 이 모델은 건축 설계, 소방 설비 관리, 화재 예방 계획 등 다양한 상황에서 활용 가능하며, 자원의 효율적 활용과 화재 위험 감소라는 중요한 목표를 달성할 수 있는 실질적인 도구로 기여할 수 있다.

3. 개발 목적

a. 머신 러닝 모델 활용 대상:

진로인 소방 설계 및 설비 전문가뿐만 아니라 소방 안전 관리자, 보험업계, 건축 설계 전문가, 공공기관을 타겟으로 머신 러닝 모델을 만들어 보았다.

b. 사용 상황:

이 모델은 소방 안전 관리와 관련된 다양한 상황에서 유용하게 사용될 수 있다. 건축 설계 단계에서는 건물의 면적, 온도, 기초 온도 등의 데이터를 기반으로 스프링클러

시스템의 설계와 설치 계획을 수립할 수 있다. 예를 들어, 특정 지역에서 건물 내부 온도가 높은 장소에는 추가적인 스프링클러를 배치해야 할 근거를 파악할 수 있다. 또한, 기존 건물의 스프링클러 시스템을 점검하고, 효율성을 평가하여 비효율적인 배치나 설치를 재설계하는 데도 활용할 수 있다.

소방 계획 수립에서도 이 모델은 중요한 역할을 한다. 화재 위험성을 예측하는 데 중요한 변수들을 식별하여 이를 기준으로 스프링클러의 개수와 배치를 최적화할 수 있다. 더 나아가, 스프링클러 설치와 유지보수에 소요되는 비용을 줄이고, 효율적인 시스템 설계와 배치를 도출함으로써 자원 활용의 효율성을 높일 수 있다.

이처럼, 이 모델은 자원의 효과적인 활용과 화재 위험 감소라는 두 가지 중요한 목표를 동시에 실현할 수 있는 모델로 활용될 수 있어 다양한 상황에서 사용될 수 있다.

c. 개발의 의의:

안전 분야, 특히 소방 설계 및 설비 분야로 진로를 설정하였고, **진로와 직접적으로 관련된 모델을 개발해보자는 생각을 기반으로 스프링클러 설계와 같은 소방 설계에 적용 가능한 머신 러닝 모델을 개발해 보았다.**

모델은 건물 내 스프링클러 설계를 최적화하기 위해 개발한 모델로, 건물의 다양한 특성과 조건(예: 면적(Surface), 온도(Temperature), 기초 온도(BaseTemperature), 거리(Distance) 등)을 분석하여 스프링클러의 최적 배치와 설계 방향을 제시하는 것을 주된 목표로 한다. 이를 통해 스프링클러 설치 및 유지보수 과정에서의 비용 절감과 자원의 효율적 관리를 실현하고, 필요 이상의 장비 설치를 방지하며 유지보수 비용도 줄일 수 있다.

d. 데이터의 어떠한 독립 변수를 사용하여 어떠한 종속 변수를 예측하는가:

스프링클러 설계 최적화를 위해 건물의 다양한 특성과 조건을 나타내는 독립 변수를 사용하여 스프링클러의 효율성과 관련된 핵심 수치를 예측한다.

독립 변수로는 건물의 면적(Surface), 실내 온도(Temperature), 기초 온도(BaseTemperature), 특정 기준 지점까지의 거리(Distance), 그리고 요일별 공휴일 여부(MondayIsDayOff ~ SundayIsDayOff) 등을 포함한다.

건물의 면적은 스프링클러 배치에 직접적인 영향을 미치며, 실내 온도와 기초 온도는 화재 발생 가능성과 밀접한 연관이 있다. 거리 변수는 스프링클러 배치 최적화를 위한 물리적 요인을 반영하며, 공휴일 변수는 건물 내 활동량의 변화를 고려하여 화재 발생 패턴을 예측하는 데 기여한다.

종속 변수는 스프링클러 설계 및 배치를 최적화하기 위한 핵심 지표인 Value 로, 화재 발생 가능성 또는 스프링클러의 효율성과 관련된 수치를 나타낸다.

모델은 데이터의 비대칭성을 완화하고 예측 성능을 높이기 위해 Value 변수의 로그 변환값(Log_Value)을 예측한다.

이를 통해 스프링클러 배치와 설계를 최적화하고, 화재 위험을 줄이며, 자원과 비용의 효율적인 활용을 도모한다.

4. 배경지식

a. 데이터 관련 사회 문제 설명:

도시화와 건물의 고층화로 인해 화재 발생 시 피해가 더욱 심각해지고 있다. 특히, 화재 발생 후 초기 대응이 늦어질 경우 인명 피해와 재산 손실이 급격히 증가할 수 있다.

많은 건물에 화재 안전 설비가 설치되어 있지만, 스프링클러와 같은 설비의 배치와 설계가 비효율적이거나 최적화되지 않은 경우가 많아 그 효과가 제한적이다.

또한, 건물의 면적, 온도, 공휴일 여부와 같은 화재 위험 요소를 분석할 수 있는 데이터가 존재하지만, 정작 이를 효과적으로 활용하는 시스템이 부족한 상황이다.

이는 화재 예방 조치가 충분히 이루어지지 않는 원인 중 하나로 작용하고 있다. 일부 지역에서는 화재 안전 규정이 엄격하지 않거나 기존 규정이 모든 건물 유형과 상황을 충분히 반영하지 못하고 있어 화재 안전 설비의 설치와 유지 관리가 미흡한 경우도 발생한다.

스프링클러와 같은 설비는 설치 및 유지보수에 큰 비용이 소요되며, 비효율적인 설계로 인해 불필요한 장비가 설치되거나 유지보수 비용이 과도하게 발생해 경제적 부담을 초래할 수 있다.

또한, 저소득층 주거지나 노후 건물처럼 화재 안전 설비가 미흡한 지역에서는 화재 발생 시 피해가 더욱 심각해질 가능성이 크다. 이는 안전에 대한 사회적 불평등 문제를 초래하며, 지역별로 상이한 화재 안전 수준이 사회적 갈등으로 이어질 수 있다.

이러한 문제를 해결하기 위해 데이터 기반의 효율적인 소방 설비 설계와 관리 방안이 필요하다.

b. 머신 러닝 모델 관련 설명 등

소방 분야에서 현재 머신 러닝은 스프링클러 시스템의 설계와 운영을 최적화하는 데 활용되고 있다. 건물 구조와 화재 위험 데이터를 학습해 스프링클러의 최적 배치 위치를 추천하며, 장비 과잉 설치를 방지하는 데 활용된다.

과거 화재 데이터를 분석해 특정 지역의 화재 발생 가능성을 예측하고, 우선적으로 스프링클러를 설치하는 데 사용된다.

실시간 센서 데이터를 분석해 온도와 연기 농도 등 이상 신호를 탐지하고, 스프링클러를 자동으로 작동시키는 데 활용되고 있다. 또한, 센서 데이터를 기반으로 고장이나 이상 동작을 사전에 감지해 유지보수 비용을 줄이는 데 기여한다.

화재 위험도와 비용 데이터를 분석해 스프링클러 설치 및 운영 비용을 최적화하는 데 사용된다. 머신 러닝은 이러한 과정을 통해 화재 피해를 최소화하고, 안전성을 향상시키는 데 중요한 역할을 하고 있다.

5. 개발 내용

a. 데이터 병합

개발 내용을 작성하기에 앞서 File 1 에는 시간(Timestamp), 온도(Temperature), 거리(Distance), 건물 사이트 ID(Siteld) 정보가 포함되어 있었고 File 2 및 File 3 에는

obs_id, SiteId, Timestamp, ForecastId, Value 컬럼이 포함되어 있었으며. File 4 에는 ForecastId 와 기간 관련 정보(ForecastPeriodNS)가 포함되어 있었다.

또한 File 5 에는 건물 ID(SiteId), 건물 면적(Surface), 샘플링 주기(Sampling), 기준 온도(BaseTemperature), 각 요일의 휴일 여부 등이 포함되어 있었고, File 6: 날짜(Date), 공휴일 이름(Holiday), SiteId 정보가 포함되어 있었다.

하지만 위 데이터들은 대부분 파일에서 열이 제대로 구분되어 있지 않았다. (: 로 구분된 형태) 이를 데이터 분석 가능한 형태로 변환하기 위해 파일별로 구분자를 처리하고 관련데이터를 ChatGPT 도움을 받아 데이터를 병합하여 데이터를 사용하였다.

b. 데이터에 대한 구체적 설명 및 시각화

i. 데이터 개수, 데이터 속성 등:

데이터셋은 총 18,849 개의 행과 20 개의 열로 구성되어 있으며, 각 행은 특정 시간 또는 기간 동안 건물의 관측 데이터를 나타낸다. 데이터는 수치형, 범주형, 그리고 날짜 데이터로 이루어져 있으며, 주요 속성은 다음과 같다.

Value 는 건물별 에너지 소비량을 나타내며, 종속 변수(종속 변수)로 설정한다. 데이터의 비대칭성을 완화하고 예측 성능을 높이기 위해 로그 변환(Log_Value)을 적용하여 모델 학습에 사용한다. **Surface** 는 건물의 면적을 나타내며, 에너지 소비량과 스프링클러 배치 전략에 영향을 미치는 변수로 활용한다.

Temperature 와 **BaseTemperature** 는 각각 현재 온도와 기준 온도를 나타낸다. 이 변수들은 에너지 소비 및 화재 발생 가능성과 밀접한 관련이 있다. **Distance** 는 특정 기준 지점까지의 거리를 나타내며, 스프링클러 배치 최적화를 위한 중요한 변수로 사용한다. 요일별 공휴일 변수(MondayIsDayOff ~ SundayIsDayOff)는 요일별 공휴일 여부를 나타내며, 건물 내 활동량 변화와 관련성을 분석하는 데 활용한다.

결측값 처리 과정에서는 **Value** 변수의 결측값을 제거하고, **Temperature** 와 **Distance** 변수의 결측값을 평균값으로 대체한다.

이를 통해 데이터 분석과 모델 학습에 적합한 형태로 데이터를 준비한다. 이러한 데이터 구조를 기반으로 모델은 건물별 에너지 소비량을 예측하며, 예측 결과를 스프링클러 설계와 배치 최적화에 적용한다.

ii. 데이터 간 상관관계:

데이터 간 상관관계는 변수들 간의 선형 관계를 평가하기 위해 상관계수를 사용하여 분석했다. 상관계수는 -1 에서 1 사이의 값을 가지며, 1 에 가까울수록 강한 양의 상관관계, -1 에 가까울수록 강한 음의 상관관계를 나타낸다.

분석후 나온 표에 따르면, Surface 와 Value 는 강한 양의 상관관계를 보이며, 건물 면적이 클수록 에너지 소비량이 증가하는 경향을 나타낸다.

Temperature 와 Value 는 음의 상관관계를 나타내며, 외부 온도가 낮을수록

에너지 소비가 증가하는 패턴이 나타난다. Distance 와 Value 는 약한 음의 상관관계를 가지며, 거리가 증가함에 따라 에너지 소비가 감소하는 경향이 약하게 나타난다.

요일별 공휴일 여부는 에너지 소비량에 약한 영향을 미치며, 특정 요일에서만 특징적인 관계가 나타날 수 있다.

iii. 시각화

시각화를 총 6 가지를 했다.

첫번째로는 Value 열의 데이터 분포를 확인하기 위해 히스토그램과 커널 밀도 추정(KDE) 그래프를 사용하여 시각화 하였다. 이는 Value 값이 특정 범위에 집중되어 있는지, 데이터가 정규 분포를 따르는 지 등을 파악할 수 있다.

두번째로는 수치형 변수 간 상관관계를 분석하였다. 데이터셋에 포함된 수치형 변수들 간의 상관관계를 히트맵으로 시각화 하였다. 히트맵은 변수 간의 상관계수를 색상으로 표현하며, 양의 상관관계와 음의 상관관계를 쉽게 확인할 수 있게 하였다. 이 시각화는 변수 선택 및 모델 해석에 유용한 정보를 제공하는데, 특히 예측 변수와 목표 변수(Value) 간의 상관성을 확인하여 모델링에 적합한 변수를 선정하는 데 도움을 준다.

세번째는 실제값과 예측값 비교(로그 변환된 값)이다. 테스트 데이터의 실제값(y_{test})과 모델이 예측한 값(y_{pred})을 비교하여 산점도로 시각화 하였다. 이 그래프는 예측값이 실제값과 얼마나 일치하는지를 시각적으로 보여준다. 이상적인 경우, 모든 점이 빨간 점선(실제값 = 예측값) 위에 위치해야 한다. 이를 통해 모델의 예측 정확성과 편향 여부를 확인하고자 하였다.

네번째는 잔차 분포 시각화이다. 잔차(실제값과 예측값의 차이)의 분포를 확인하기 위해 히스토그램과 커널 밀도 추정 그래프를 생성하였다. 잔차가 정규 분포에 가까운 형태를 보인다면 모델이 데이터를 잘 설명하고 있음을 의미한다. 또한, 잔차의 평균이 0 에 가까운지, 특정 방향으로 편향되어 있는지 확인함으로써 모델의 성능을 분석하고자 하였다.

다섯번째는 입력 변수 중요도 시각화이다. 랜덤 포레스트 모델이 계산한 입력 변수들의 중요도를 막대 그래프로 시각화 하였다. 이 그래프는 모델이 예측 과정에서 각 변수를 얼마나 활용했는지를 보여준다. 이는 예측에 가장 중요한 변수를 식별할 수 있으며, 시스템 설계 및 최적화에 활용할 수 있는 기초 데이터를 제공한다.

여섯번째는 K-Fold 교차검증 결과로 Fold 에서의 평균 MSE 와 R^2 점수를 시각화하여 모델의 일관성과 성능을 평가한다.

c. 데이터에 대한 설명 이후, 어떤 것을 예측하고자 하는지 구체적으로 설명

i. 종속변수, 독립변수 설정:

종속변수로는 건물별 에너지 소비량(Value)로 잡았다.

```
final_merged_data["Log_Value"] = np.log1p(final_merged_data["Value"])
y = final_merged_data["Log_Value"]
```

건물별 에너지 소비량은 건물이 사용하는 총 에너지 즉 건물 내 활동 수준 또는 점유율과 연관 될 뿐더러 높은 소비량은 인구 밀도나 활동량이 많음을 나타낼 수도 있다. 또한 로그 변환된 값을 예측 대상으로 삼아 데이터의 비대칭성을 완화하고 모델의 성능을 높이하고자 하였다.

```
features = ["Surface", "Temperature", "BaseTemperature", "Distance",
            "MondayIsDayOff", "TuesdayIsDayOff", "WednesdayIsDayOff",
            "ThursdayIsDayOff", "FridayIsDayOff", "SaturdayIsDayOff", "SundayIsDayOff"]
X = final_merged_data[features]
```

독립변수는 건물의 면적, 온도, 거리, 휴일여부 등으로 에너지 소비량에 영향을 미칠 수 있는 변수들로 구성되었다. 이러한 설정을 통해 건물의 에너지 소비량을 예측하고 이를 기반으로 스프링클러 설계와 배치를 최적화하고자 하였다.

d. 머신 러닝 모델 선정 이유

i. 설명한 데이터를 기반으로 머신 러닝 모델 선정 이유:

랜덤 포레스트 회귀(Random Forest Regressor)는 본 데이터의 특성과 예측 목표에 적합한 머신 러닝 모델로 적합하다 판단하여 선정하였다.

랜덤 포레스트는 여러 개의 의사결정 트리를 결합하여 예측 성능을 향상시키는 앙상블 기법으로, 수치형 변수와 범주형 변수가 혼합된 데이터셋에서 특히 강점을 보인다.

데이터셋은 건물의 면적(Surface), 온도(Temperature), 공휴일 여부(MondayIsDayOff ~ SundayIsDayOff)와 같은 수치형 및 범주형 변수를 포함하고 있어, 랜덤 포레스트의 장점을 잘 활용 가능하다.

랜덤 포레스트는 비선형 관계를 포함한 복잡한 데이터 패턴을 효과적으로 학습하며, 범주형 변수와 수치형 변수 간의 관계를 동시에 처리할 수 있다.

또한, 일부 결측값이나 이상치가 포함된 데이터에서도 높은 성능을 유지하는 강점을 보인다.

특히, 랜덤 포레스트는 각 변수의 중요도를 계산하여 예측에 가장 큰 영향을 미치는 변수를 식별할 수 있으며, 이는 스프링클러 설계 최적화를 위한 주요 변수를 탐색하는 데 중요한 역할을 한다고 생각했기 때문이다.

따라서 랜덤 포레스트 회귀는 데이터셋의 특성과 예측 목표에 부합하는 최적의 모델로 판단되어, 이를 활용하여 데이터 기반의 스프링클러 설계 및 배치 전략을 수립하고자 하였다.

ii. 성능 비교를 위한 머신 러닝 모델 선정 이유

위에서 언급한 바와 같이 모델 선정에 있어, 나는 랜덤 포레스트 회귀(Random Forest Regressor)를 선택했다.

이 모델을 선택한 이유는, 내가 다루는 소방 설계 및 설비 분야의 데이터가

다소 복잡하고 다양한 변수들 간에 비선형적인 관계가 존재하기 때문이다.

랜덤 포레스트는 이러한 복잡한 패턴을 잘 학습할 수 있으며, 변수 선택에 있어 중요한 특징을 자동으로 추출할 수 있는 능력이 있다. 또한, 모델이 여러 트리로 구성되어 있기 때문에 개별 트리의 편향을 줄이고, 예측의 정확성을 높일 수 있다.

또한, 랜덤 포레스트는 학습 속도가 비교적 빠르고, 하이퍼파라미터 튜닝이 비교적 직관적이어서 실험과 평가 과정에서 효율적으로 활용할 수 있었다.

이러한 이유로, 랜덤 포레스트 회귀 모델은 나의 프로젝트에서 매우 적합한 선택이었으며, 다른 회귀 모델들과의 성능 비교를 통해 모델의 강점을 잘 평가할 수 있을 것으로 판단했다.

e. 사용할 성능 지표

i. 머신 러닝 모델의 성능을 평가하기 위해 사용하는

성능 지표에 관한 설명 :

머신 러닝 모델을 성능을 평가하기 위해 사용할 성능 지표는 **평균 제곱 오차(Mean Squared Error), 결정계수(R^2), RMSE, 잔차 분석 K-fold** 이다.

평균 제곱 오차(MSE)는 모델이 예측한 값과 실제 값 간의 차이를 제곱하여 평균을 낸 값으로 이 값은 예측의 오차 크기를 정량적으로 나타낸다. MSE 값이 작을수록 모델의 예측이 실제 값에 더 가까워지며, 모델의 성능이 좋다고 평가할 수 있다. 그러나 MSE 는 오차가 클수록 그 영향이 더 커지기 때문에 극단적인 오류가 있는 경우 더 큰 패널티를 부여한다.

결정계수(R^2)은 회귀 모델이 데이터의 분산을 얼마나 잘 설명하는지를 나타내는 지표이다. R^2 값은 0 과 1 사이의 값을 가지며, 1 에 가까울수록 모델이 데이터를 잘 설명한다고 할 수 있다. R^2 값이 0 이라면 모델이 데이터를 전혀 설명하지 못한다는 의미이며, 음수일 경우 모델이 매우 부정확하다는 의미이다. 잔차 분석은 모델이 예측하지 못한 패턴을 감지하는데 유용하며 모델이 특정 패턴에 대해 과도하게 잘못 예측하는지, 아니면 특정 범위의 데이터에서 성능이 떨어지는지 파악할 수 있다. 즉, 잔차 분석을 통해 모델의 과적합 여부나 데이터의 이상값을 확인할 수 있어, 예측의 정확도를 더욱 높일 수 있다.

RMSE 는, 평균 제곱근 오차를 의미한다. 이는 회귀 모델의 성능을 평가할 때 자주 사용되는 지표 중 하나이다. RMSE 값이 작을수록 모델의 예측이 실제값에 가까운 것을 의미한다.

잔차분석은 회귀 분석에서 모델의 성능을 평가하고, 모델이 데이터를 얼마나 잘 설명하는지, 그리고 모델이 놓친 패턴이나 문제가 있는지 파악하기 위해 사용되는 기법이다.

K-Fold 교차 검증은 모델의 일반화 성능을 평가하는 데 매우 중요한 방법이다. K-Fold 교차 검증은 데이터를 K 개의 폴드(fold)로 나누고, 각 폴드에 대해 모델을

학습하고 평가하는 과정을 반복한다. 이 과정에서 모델은 여러 번 학습되고 평가되며, 각 Fold 에서 계산된 성능 지표(MSE 와 R^2)의 평균을 통해 모델의 성능을 평가할 수 있다. K-Fold 교차 검증은 모델이 특정 데이터셋에 과적합되지 않고, 다양한 데이터에 대해 잘 일반화되는지 확인하는 데 유용하다. 또한, K-Fold 교차 검증을 통해 모델이 다양한 데이터셋에서 어떻게 동작하는지 평가할 수 있기 때문에, 모델의 일관성과 신뢰성을 높일 수 있다.

ii. 성능 지표 선정 이유:

위에서 언급한 성능 지표인 **평균 제곱 오차(MSE)**, **결정계수(R^2)**, **RMSE**, **잔차 분석**, 그리고 **K-Fold 교차 검증**은 **회귀 모델의 성능을 평가하는 데 적합한 지표들**이다.

평균 제곱 오차(MSE)는 예측값과 실제값 간의 차이를 정량적으로 나타내며, 모델이 얼마나 정확하게 예측하는지를 평가하는 지표이다. 값이 작을수록 예측이 정확하다는 의미이다.

결정계수(R^2)는 모델이 데이터를 얼마나 잘 설명하는지, 즉 데이터의 분산을 얼마나 잘 설명하는지를 평가하는 지표로, 모델의 설명력을 측정하는 데 유용하다. R^2 값은 1 에 가까울수록 모델이 데이터를 잘 설명한다고 할 수 있다.

RMSE (Root Mean Squared Error)는 평균 제곱 오차(MSE)의 제곱근으로, 예측값과 실제값 간의 차이를 원래 데이터의 단위로 표현한 지표이다. RMSE 는 직관적으로 예측의 정확도를 평가하는 데 유용하며, 값이 작을수록 예측이 정확하다는 의미이다. RMSE 는 오차가 클수록 더 큰 패널티를 부여하므로, 큰 오차가 발생한 경우 이를 쉽게 파악할 수 있다. 또한, RMSE 는 원래 데이터의 단위와 동일한 단위를 가지기 때문에, 예측값과 실제값 간의 차이를 실제 단위로 해석할 수 있어 직관적인 평가가 가능하다.

잔차 분석은 모델의 예측값과 실제값 간의 차이인 잔차를 분석하여 모델의 성능을 평가하는 방법이다. 잔차가 랜덤하게 분포한다면 모델이 적절하게 학습되었다고 할 수 있으며, 특정 패턴이 보인다면 모델이 잘못 학습되었음을 나타낸다.

K-Fold 교차 검증은 데이터를 여러 개의 폴드로 나누어 모델을 평가하고, 모델의 일반화 성능을 확인하는 방법이다. 교차 검증을 통해 모델이 훈련 데이터에 과적합(overfitting)되지 않았는지 확인할 수 있다.

따라서 **평균 제곱 오차(MSE)**, **결정계수(R^2)**, **잔차 분석**, 그리고 **K-Fold 교차 검증**을 종합적으로 활용하면 **모델의 정확성을 높이고, 모델이 다양한 데이터셋에서 어떻게 동작하는지 평가할 수 있기 때문에 스프링클러 설계 최적화 모델의 성능을 더욱 향상시킬 수 있다.** 이러한 이유로 이 성능 지표들을 선정했다.

6. 개발 과정

```
7. import os
import pandas as pd # 데이터 처리
import numpy as np # 수치 계산
from sklearn.model_selection import train_test_split # 데이터 분할
from sklearn.ensemble import RandomForestRegressor # 랜덤 포레스트 모델
from sklearn.metrics import mean_squared_error, r2_score # 모델 평가 지표
import matplotlib.pyplot as plt # 시각화
import seaborn as sns # 시각화

# Step 1: 상대 경로 설정
file_path = "./기말데이터/1~6 파일_정리_파일.csv" # 상대 경로 설정

# Step 2: 데이터 로드
try:
    final_merged_data = pd.read_csv(file_path) # 데이터 읽기
    print("데이터 로드 완료! 데이터 크기:", final_merged_data.shape)
except FileNotFoundError:
    print(f"파일을 찾을 수 없습니다: {file_path}")
    raise

# Step 3: 결측값 처리
# 'Value' 열의 결측값 제거 (종속 변수)
final_merged_data = final_merged_data.dropna(subset=["Value"])
# 'Temperature', 'Distance'의 결측값을 평균으로 대체
final_merged_data["Temperature"] =
final_merged_data["Temperature"].fillna(final_merged_data["Temperature"].mean())
final_merged_data["Distance"] =
final_merged_data["Distance"].fillna(final_merged_data["Distance"].mean())

# Step 4: 로그 변환 (종속 변수)
# 'Value'의 비대칭 분포를 완화하기 위해 로그 변환하였음
final_merged_data["Log_Value"] = np.log1p(final_merged_data["Value"])

# Step 5: 입력 변수(X)와 타겟 변수(y) 준비
# 입력 변수
features = ["Surface", "Temperature", "BaseTemperature", "Distance",
```

```

        "MondayIsDayOff", "TuesdayIsDayOff", "WednesdayIsDayOff",
        "ThursdayIsDayOff", "FridayIsDayOff", "SaturdayIsDayOff", "SundayIsDayOff"]
X = final_merged_data[features] # 입력 변수
y = final_merged_data["Log_Value"] # 타겟 변수

# Step 6: 학습 데이터와 테스트 데이터 분리
# 데이터 분할 (80% 학습, 20% 테스트)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 7: 랜덤 포레스트 회귀 모델 학습
model = RandomForestRegressor(n_estimators=100, random_state=42) # 100 개의 트리 사용
model.fit(X_train, y_train) # 모델 학습

# Step 8: 모델 평가
# 테스트 데이터를 사용해 예측
y_pred = model.predict(X_test)
# 평균 제곱 오차(MSE)와 결정 계수(R²) 계산
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("평균 제곱 오차 (MSE):", mse)
print("결정 계수 (R²):", r2)

# Step 9: 실제값 vs 예측값 시각화
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.7, color="blue")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title("실제값 vs 예측값 (로그 변환된 값)")
plt.xlabel("실제 로그 값")
plt.ylabel("예측 로그 값")
plt.show()

# Step 10: 잔차 분석
# 잔차 계산 (실제값 - 예측값)
residuals = y_test - y_pred
plt.figure(figsize=(8, 6))
sns.histplot(residuals, kde=True, color="purple", bins=30)
plt.title("잔차 분포")

```

```

plt.xlabel("잔차 (실제값 - 예측값)")
plt.ylabel("빈도")
plt.show()

# Step 11: 변수 중요도 시각화
# 랜덤 포레스트 모델의 변수 중요도 계산
feature_importances = model.feature_importances_
importance_df = pd.DataFrame({
    "Feature": features,
    "Importance": feature_importances
}).sort_values(by="Importance", ascending=False)

# 변수 중요도 시각화
plt.figure(figsize=(10, 6))
sns.barplot(x="Importance", y="Feature", data=importance_df, palette="viridis")
plt.title("입력 변수 중요도")
plt.xlabel("중요도")
plt.ylabel("입력 변수")
plt.show()

```

초기 소스코드이다. 시각화를 하여 그래프를 나오게 하는것은 성공하였지만 한글 폰트가 지원이 되지 않아 소스코드 수정을 필요로 하였다.

한글 폰트 설정 ---> 한글 폰트를 설정을 안하니 그래프 나올때 오류가 발생하여 한글 폰트 설정하였습니다.

```

plt.rcParams['font.family'] = 'Malgun Gothic' # '맑은 고딕' 폰트 설정
plt.rcParams['axes.unicode_minus'] = False # 음수 기호 제대로 표시하기 위해 설정하였다.

```

위와 같은 소스코드를 추가하여 문제를 개선하였다. 하지만 이번에는 이 모델이 데이터를 잘 요약 정리를 하고 시각화 그래프가 잘 되어 있는지를 봤지만 실제값과 예측값 비교, 잔차 분포 그래프, 변수 중요도 막대 그래프는 생성되었지만 수치 형 변수간 상관관계와 종속 변수 분포를 확인하기 위한 시각화 자료는 없어 이를 추가하기 위한 소스 코드 추가 필요성을 느껴 머신러닝 모델의 소스코드를 수정해 보았다.

```

# Step 2: 데이터 요약 및 탐색
print("\n 데이터 요약:")
print(final_merged_data.info()) # 데이터 타입 및 결측값 확인
print("\n 기본 통계 요약:")
print(final_merged_data.describe()) # 기본 통계 요약

```

이를 수정한 구체적 이유는 모델링하기 전에 데이터의 구조와 특성을 이해하고, 전처리 방향을 설계하기 위해서이다.

.info()는 데이터 타입, 결측값 개수, 열 이름 등을 확인하여 데이터의 품질과 결측값 처리 필요성을 파악할 수 있게 한다.

.describe()는 수치형 변수의 요약 통계를 제공하여 데이터의 분포와 변동성을 이해하고, 이상치를 탐지하거나 변환 필요성을 평가하는 데 도움을 주기 때문이다.

```
# 데이터 분포 시각화 (Value, Temperature, Distance)
plt.figure(figsize=(8, 6))
sns.histplot(final_merged_data["Value"].dropna(), kde=True, bins=30, color="blue")
plt.title("Value 분포")
plt.xlabel("Value")
plt.ylabel("빈도")
plt.show()
```

종속 변수(Value)의 분포를 확인하기 위해 히스토그램 시각화를 추가하였다. 분포를 통해 데이터의 비대칭성 및 이상치를 확인하고 로그 변환이 왜 필요한지 근거를 제공한다.

```
# 상관관계 분석 (수치형 데이터만)
numeric_data = final_merged_data.select_dtypes(include=[np.number])
plt.figure(figsize=(10, 8))
sns.heatmap(numeric_data.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("수치형 변수 간 상관관계")
plt.show()
```

이는 모든 수치형 변수간 상관관계를 확인하기 위해 히트맵 시각화를 추가하였다. 독립변수와 종속변수 간의 연관성을 파악하여, 예측에 중요한 변수를 식별하고자 했다.

이러한 수정을 통해 데이터에 대한 초기 탐색적 분석을 강화하였으며, 데이터 품질 확인, 모델 성능 향상 기반 마련을 하였다.

```
plt.show()
```

plt.show()로 소스코드를 만들었는데 이는 화면에 일시적으로 그래프를 표시할 뿐 기록에 남지 않아, 재사용이나 확인이 어렵다. 따라서 재사용성, 보고서 작성, 그리고 자동화된 분석 파이프라인 구축에 유리하기 위해 그래프를 저장하고자 하였다.

```
plt.savefig()
```

위 코드를 사용해 그래프를 저장하여 분석과정에서 생성된 결과를 파일로 보관하였다. 이로 인해 저장된 그래프는 나중에 분석과정을 복기하거나, 다른 프로젝트에서 다시 활용이 가능하다.

```

# 라이브러리 불러오기
import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

# 한글 폰트 설정 ----> 초기 소스코드에서 나온 그래프에서 폰트 문제가 생겨 위와 같은
소스코드를 추가
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

# 절대 경로가 아닌 상대 경로 설정 및 데이터 로드하였음
file_path = "../기말데이터/1~6 파일_정리_파일.csv"

try:
    final_merged_data = pd.read_csv(file_path) # 데이터 읽기
    print("데이터 로드 완료! 데이터 크기:", final_merged_data.shape)
except FileNotFoundError:
    print(f"파일을 찾을 수 없습니다: {file_path}")
    raise

# 데이터 요약 및 탐색
print("\n 데이터 요약:")
print(final_merged_data.info()) # 데이터 타입 및 결측값 확인
print("\n 기본 통계 요약:")
print(final_merged_data.describe()) # 기본 통계 요약

# 데이터 분포 시각화 (Value, Temperature, Distance)
plt.figure(figsize=(8, 6))
sns.histplot(final_merged_data["Value"].dropna(), kde=True, bins=30,
color="blue")
plt.title("Value 분포")
plt.xlabel("Value")
plt.ylabel("빈도")
plt.savefig("./value_distribution.png") # 그래프 저장
plt.close() # 메모리 해제

# 상관관계 분석 (수치형 데이터만)
numeric_data = final_merged_data.select_dtypes(include=[np.number])
plt.figure(figsize=(10, 8))
sns.heatmap(numeric_data.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("수치형 변수 간 상관관계")
plt.savefig("./correlation_plot.png") # 그래프 저장
plt.close() # 메모리 해제

# 결측값 처리
final_merged_data = final_merged_data.dropna(subset=["Value"]) # 'Value' 열의
결측값 제거
final_merged_data["Temperature"] =

```

```

final_merged_data["Temperature"].fillna(final_merged_data["Temperature"].mean())
final_merged_data["Distance"] =
final_merged_data["Distance"].fillna(final_merged_data["Distance"].mean())

# 로그 변환 (종속 변수)
final_merged_data["Log_Value"] = np.log1p(final_merged_data["Value"])

# 입력 변수(X)와 종속 변수(y) 준비
features = ["Surface", "Temperature", "BaseTemperature", "Distance",
            "MondayIsDayOff", "TuesdayIsDayOff", "WednesdayIsDayOff",
            "ThursdayIsDayOff", "FridayIsDayOff", "SaturdayIsDayOff",
            "SundayIsDayOff"]
X = final_merged_data[features]
y = final_merged_data["Log_Value"]

# 학습 데이터와 테스트 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# 랜덤 포레스트 회귀 모델 학습
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 모델 평가
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse) # RMSE 계산
r2 = r2_score(y_test, y_pred)
print("\n 모델 평가 결과:")
print("평균 제곱 오차 (MSE):", mse)
print("RMSE (Root Mean Squared Error):", rmse)
print("결정 계수 (R²):", r2)

# 실제값 vs 예측값 시각화
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.7, color="blue")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',
         lw=2)
plt.title("실제값 vs 예측값 (로그 변환된 값)")
plt.xlabel("실제 로그 값")
plt.ylabel("예측 로그 값")
plt.savefig("./actual_vs_predicted.png") # 그래프 저장
plt.close()

# 잔차 분석
residuals = y_test - y_pred
plt.figure(figsize=(8, 6))
sns.histplot(residuals, kde=True, color="purple", bins=30)
plt.title("잔차 분포")
plt.xlabel("잔차 (실제값 - 예측값)")
plt.ylabel("빈도")
plt.savefig("./residuals_distribution.png") # 그래프 저장
plt.close()

```

```

# 변수 중요도 시각화
feature_importances = model.feature_importances_
importance_df = pd.DataFrame({
    "Feature": features,
    "Importance": feature_importances
}).sort_values(by="Importance", ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(x="Importance", y="Feature", data=importance_df,
palette="viridis", hue="Feature")
plt.title("입력 변수 중요도")
plt.xlabel("중요도")
plt.ylabel("입력 변수")
plt.savefig("./feature_importance.png") # 그래프 저장
plt.close()

from sklearn.model_selection import KFold, cross_val_score

# K-Fold 교차 검증 설정
kf = KFold(n_splits=5, shuffle=True, random_state=42) # 5-Fold 교차 검증 설정

# MSE, RMSE, R^2 점수 저장
mse_scores = []
rmse_scores = [] # RMSE 저장 리스트 추가
r2_scores = []

# K-Fold로 데이터 나누고 모델 평가
for train_index, test_index in kf.split(X):
    X_train_kf, X_test_kf = X.iloc[train_index], X.iloc[test_index]
    y_train_kf, y_test_kf = y.iloc[train_index], y.iloc[test_index]

    # 모델 학습
    model.fit(X_train_kf, y_train_kf)

    # 예측 및 평가
    y_pred_kf = model.predict(X_test_kf)
    mse_fold = mean_squared_error(y_test_kf, y_pred_kf)
    rmse_fold = np.sqrt(mse_fold) # RMSE 계산
    mse_scores.append(mse_fold)
    rmse_scores.append(rmse_fold) # RMSE 저장
    r2_scores.append(r2_score(y_test_kf, y_pred_kf))

# 평균 성능 출력
mean_mse = np.mean(mse_scores)
mean_rmse = np.mean(rmse_scores) # 평균 RMSE 계산
mean_r2 = np.mean(r2_scores)
print("\nK-Fold 교차 검증 결과:")
print(f"평균 MSE: {mean_mse:.3f}")
print(f"평균 RMSE: {mean_rmse:.3f}")
print(f"평균 R^2: {mean_r2:.3f}")

# 시각화: Fold 별 MSE, RMSE, R^2 점수

```

```
plt.figure(figsize=(10, 6))
plt.plot(range(1, kf.get_n_splits() + 1), mse_scores, marker='o',
label='MSE', color='blue')
plt.plot(range(1, kf.get_n_splits() + 1), rmse_scores, marker='o',
label='RMSE', color='red')
plt.plot(range(1, kf.get_n_splits() + 1), r2_scores, marker='o', label='R^2',
color='green')
plt.title('K-Fold 교차 검증 결과 (MSE, RMSE, R^2)')
plt.xlabel('Fold')
plt.ylabel('Score')
plt.legend()
plt.savefig("./kfold_results_rmse.png") # 그래프 저장
plt.close()
```

마지막으로 모델이 얼마나 예측을 잘하고 있는지 구체적으로 확인하기 위해서
수치자료인 RMSE 및 K-Fold 를 추가한 최종 완성본이다.

```
C:\Users\User\AppData\Local\Programs\Python\Python38\python.exe "C:\Users\User\Documents\GitHub\01_Python_and_Git\데이터 프레임\기말고사\기말고사 보고서\기말보고서(Sprinkler Sense).py"
데이터 로드 완료! 데이터 크기: (18849, 20)

데이터 요약:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18849 entries, 0 to 18848
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Unnamed: 0            18849 non-null  int64
1   Timestamp             18849 non-null  object
2   Temperature           18849 non-null  float64
3   Distance              18849 non-null  float64
4   SiteId                18849 non-null  int64
5   Date                  18849 non-null  object
6   obs_id                18849 non-null  int64
7   ForecastId            18849 non-null  int64
8   Value                 18509 non-null  float64
9   Surface               18849 non-null  float64
10  Sampling              18849 non-null  int64
11  BaseTemperature       18849 non-null  int64
12  MondayIsDayOff        18849 non-null  bool
13  TuesdayIsDayOff       18849 non-null  bool
14  WednesdayIsDayOff     18849 non-null  bool
15  ThursdayIsDayOff      18849 non-null  bool
16  FridayIsDayOff        18849 non-null  bool
17  SaturdayIsDayOff      18849 non-null  bool
18  SundayIsDayOff        18849 non-null  bool
19  Holiday               1 non-null      object
dtypes: bool(7), float64(4), int64(6), object(3)
memory usage: 2.0+ MB
None
```

기본 통계 요약:

	Unnamed: 0	Temperature	...	Sampling	BaseTemperature
count	18849.000000	18849.000000	...	18849.000000	18849.0
mean	9424.000000	13.292090	...	24.987532	18.0
std	5441.381948	6.424836	...	7.967437	0.0
min	0.000000	-10.600000	...	5.000000	18.0
25%	4712.000000	9.000000	...	15.000000	18.0
50%	9424.000000	13.100000	...	30.000000	18.0
75%	14136.000000	17.000000	...	30.000000	18.0
max	18848.000000	37.000000	...	30.000000	18.0

[8 rows x 10 columns]

모델 평가 결과:

평균 제곱 오차 (MSE): 0.5563795381705279

RMSE (Root Mean Squared Error): 0.7459085320403085

결정 계수 (R^2): 0.725860951582964

K-Fold 교차 검증 결과:

평균 MSE: 0.560

평균 RMSE: 0.749

평균 R^2 : 0.730

종료 코드 0(으)로 완료된 프로세스

다음은 머신 러닝 모델 실행 결과값이다.

7. 개발 결과

A. 성능 지표에 따른 머신 러닝 모델 성능 평가

i. 수치 자료 및 시각화 자료를 사용

1. MSE, RMSE, 결정계수(R^2)

MSE 값은 0.556, RMSE 값은 0.746 이며 결정계수(R^2)값은 0.726

MSE (0.5564)와 RMSE (0.746)는 모두 모델의 예측 오차를 나타낸다.

RMSE 는 예측 오차의 크기를 원래 데이터의 단위로 보여주기 때문에 얼마나 잘 예측했는지를 더 직관적으로 이해할 수 있다.

R^2 값은 0.7259 로, 모델이 데이터의 약 73%의 분산을 설명하고 있다는 것을 의미한다. 이는 모델이 상당히 좋은 성능을 보이고 있다는 지표이다.

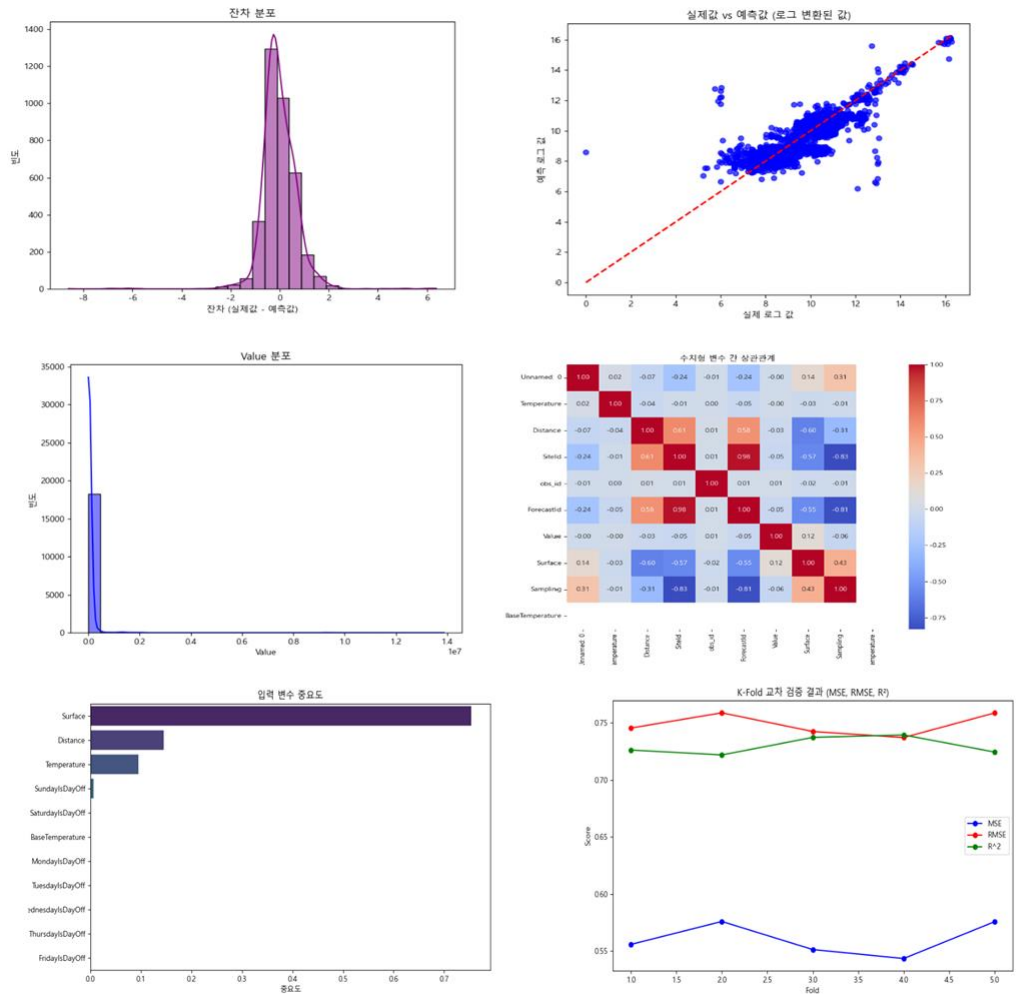
이 RMSE 값과 MSE 값은 모델이 예측을 잘 수행하고 있다는 것을 보여주며, 모델이 실제값과 예측값 간의 차이를 잘 줄였음을 시사한다.

2. 시각화 자료

잔차분포(히스토그램), 실제값 vs 예측값(산점도), value 분포(히스토그램),

수치형 변수 간 상관관계(히트맵), 변수 중요도 (막대 그래프), K-Fold 교차 검증

결과(선 그래프)



3. KFold 결과

평균 MSE 는 K-Fold 교차 검증 결과에서 0.560 으로 나타났다. 이는 모델이 다양한 데이터셋에 대해 일관되게 잘 예측하고 있다는 것을 의미한다. MSE 값이 평균적으로 0.560 으로 안정적이라면, 모델이 과적합되지 않고 일반화 성능이 좋은 것으로 평가할 수 있다.

평균 R² 값은 0.730 으로, 모델이 K-Fold 교차 검증을 통해 여러 번 평가되었을 때 평균적으로 약 73%의 분산을 설명한다는 뜻이다. 이는 모델이 여러 데이터셋에서 일관되게 좋은 성능을 보이고 있다는 것을 의미한다.

평균 RMSE 는 0.749 로 나타났다. RMSE 는 예측값과 실제값 간의 차이를 원래 데이터의 단위로 표현한 지표로, 값이 작을수록 예측이 정확하다는 의미이다. 평균 RMSE 값이 0.749 라면, 모델이 예측한 값과 실제 값 간의 오차가 평균적으로 약 0.749 정도라는 것을 의미하며, 이는 예측 정확도가 꽤 높은 수준임을 나타낸다.

ii. 다른 머신 러닝 모델과 성능 비교

B. 머신 러닝 모델의 성능 결과에 대한 해석:

모델의 성능을 평가한 결과, 평균 제곱 오차(MSE)는 0.556, 루트 평균 제곱 오차(RMSE)는 0.746 으로 나타났다. 이 값들은 모델의 예측 오차를 나타내며, RMSE 값이 상대적으로 더 직관적이므로 예측 성능을 평가할 때 유용하다. RMSE 값이 낮을수록 모델이 실제 값을 잘 예측하고 있다는 의미이다. 또한, 결정 계수(R^2) 값은 0.726 으로, 모델이 데이터의 약 73%의 분산을 설명하고 있음을 보여준다. 이는 모델이 상당히 좋은 성능을 보이고 있다는 지표이다.

K-Fold 교차 검증 결과에서도 평균 MSE 는 0.560, 평균 R^2 는 0.73, 평균 RMSE 는 0.749 로 나타났다. 이는 모델이 다양한 데이터셋에 대해 일관되게 잘 예측하고 있다는 것을 의미하며, 모델이 과적합되지 않고 일반화 성능이 좋은 것으로 평가할 수 있다. R^2 값은 모델이 실제 값과 예측 값 간의 관계를 잘 설명하고 있음을 보여준다.

시각화 자료를 통해 모델의 성능을 더욱 명확하게 파악할 수 있었다. Value 분포를 나타내는 히스토그램 실제값과 예측값 간의 관계를 나타내는 산점도는 모델이 예측을 잘 하고 있다는 것을 의미하였고. 잔차 분포 히스토그램은 정규분포에 가까워 모델이 잘 학습되었음을 나타냈다, 수치형 변수 간의 상관관계 히트맵은 변수 간의 관계를 시각적으로 파악하는 데 유용했다. 변수 중요도 그래프는 예측에 중요한 변수들을 식별하는 데 도움을 주었으며, K-Fold 교차 검증 결과는 모델이 여러 데이터셋에서 일관되게 좋은 성능을 보였음을 확인할 수 있었다.

8. 결론

A. 머신 러닝 모델 개발에 관한 간략한 요약 및 결과 설명

진로 희망 분야인 소방 설계 및 설비 분야에서 활용할 수 있는 머신 러닝 모델을 개발하여 건물 내 스프링클러 설계를 최적화하는 데 중점을 두었다.

모델은 Kaggle 의 **Buildings Energy Consumption Dataset** 을 기반으로, 건물의 면적(Surface), 온도(Temperature), 기준 온도(BaseTemperature), 거리(Distance), 공휴일 여부(MondayIsDayOff ~ SundayIsDayOff) 등 다양한 독립 변수를 활용하여 스프링클러의 최적 배치와 설계를 예측한다.

이 모델은 건물별 에너지 소비량(Value)을 예측하는데 중점을 두었으며, 예측 결과는 스프링클러 설계 최적화와 비용 절감, 자원 효율성을 높이는 데 기여할 수 있다.

모델은 **랜덤 포레스트 회귀(Random Forest Regressor)** 알고리즘을 사용하여 데이터의 비선형 관계와 변수 간 복잡한 상호작용을 처리했다.

모델의 성능은 **평균 제곱 오차(MSE)**, **결정계수(R^2)**, **잔차 분석**, **K-Fold 교차 검증** 등을 통해 평가되었으며, 예측 정확도와 모델의 일반화 성능이 우수한 결과를 보였다.

이를 통해 스프링클러 설계 및 배치 최적화에서 중요한 역할을 할 수 있는 실용적인 도구를 제공하였다.

B. 개발 의의 등

이 모델의 개발은 소방 설계 및 설비 분야의 실용적인 문제를 해결하는 데 중요한 기여를 한다.

특히, 스프링클러 설계 최적화를 통해 건물의 화재 예방 및 안전성을 높이고, 자원의 효율적 사용과 유지보수 비용 절감을 도모할 수 있다.

또한, 이 모델은 소방 설계 전문가, 건축 설계 전문가, 소방 안전 관리자 등 다양한 분야의 전문가들이 활용할 수 있는 도구로, 안전 관리 및 설계 과정에서 중요한 결정을 내리는 데 실질적인 도움이 될 것이다.

이번 프로젝트는 소방 설계 및 안전 관리 분야에 머신 러닝을 적용하여 새로운 접근 방식을 제시하며, 데이터 기반의 최적화 모델이 현장에 어떻게 적용될 수 있는지에 대한 중요한 사례를 제공한다.

또한, 머신 러닝 모델을 통해 설계 및 배치 최적화뿐만 아니라, 화재 발생 가능성을 예측하고 이에 따른 예방 조치를 강화하는 데 기여할 수 있다.

C. 개발한 머신 러닝 모델의 한계

본 모델은 여러 가지 변수들을 기반으로 예측을 수행하지만, 여전히 몇 가지 한계가 존재한다.

첫째, 데이터의 품질이나 정확성에 따라 모델의 성능이 달라질 수 있다. 예를 들어, 결측값 처리나 이상치가 모델 성능에 영향을 미칠 수 있다.

둘째, 랜덤 포레스트 모델은 복잡한 데이터 패턴을 잘 학습하지만, 과적합(overfitting)의 위험이 있을 수 있다. 이를 해결하기 위해 K-Fold 교차 검증을 사용하여 성능을 평가했지만, 여전히 일부 데이터에서는 예측 정확도가 떨어질 수 있다.

셋째, 모델은 특정 변수들에 의존하고 있기 때문에, 다른 변수들이 추가되었을 때 예측 성능이 변화할 수 있다. 향후 모델 개선을 위해 추가적인 변수나 다른 머신 러닝 기법을 고려할 필요가 있다.

9. 느낀점

머신 러닝 모델을 개발하면서 여러 가지 중요한 점을 배울 수 있었다. 먼저, 머신 러닝이 실제 문제를 해결하는 데 강력한 도구라는 점을 실감했다. 특히, 소방 설계 및 설비 분야에 모델을 적용하면서 데이터 기반 접근 방식이 설계 최적화와 자원 절감에 크게 기여할 수 있다는 점이 매우 인상적이었다.

데이터의 품질과 다양성이 모델 성능에 얼마나 큰 영향을 미치는지도 배웠다. 데이터를 깊이 이해하고 처리하는 과정이 모델 개발의 핵심이라는 것을 깨닫게 되었고, 결측값 처리와 변수 선택이 모델 성능에 얼마나 중요한 요소인지 실감했다.

MSE, RMSE, R^2 등 성능 지표를 활용하면서 모델의 강점과 약점을 정량적으로 평가할 수 있다는 점이 유익했다. 특히, 랜덤 포레스트 모델이 변수 간 복잡한 상호작용을 학습하는 데 효과적이라는 점을 경험하며, 문제에 적합한 알고리즘 선택이 얼마나 중요한지 알게 되었다. 또한, 분야의 전문적 지식이 모델 개발 과정에서 큰 역할을 한다는 것을 느꼈다. 소방 설계와 관련되어 찾아본 배경 지식이 변수 선택과 모델 해석에 큰 도움을 주었다. 하지만 모델 성능이

여전히 데이터와 변수에 크게 의존한다는 점에서, 새로운 변수 추가와 다양한 알고리즘 실험의 필요성을 절실히 느꼈다.

결론적으로, 머신 러닝 모델 개발은 단순히 알고리즘을 적용하는 과정이 아니라 데이터 이해, 모델 설계, 성능 평가, 그리고 전문적 지식을 결합하는 종합적인 과정이라는 것을 깨달았다. 반복적인 실험과 학습을 통해 점진적으로 발전할 수 있다는 점이 매우 의미 있었다.

이는 앞으로 소방 설계 및 설비 엔지니어를 희망하는 나에게 있어 매우 값진 경험이 되었다. 이번 프로젝트를 통해 단순히 이론에 머무르지 않고, 실제 데이터를 활용하여 문제를 해결하는 과정을 직접 체험할 수 있었다. 특히, 머신 러닝 모델을 설계하고 평가하는 과정에서 얻은 통찰은 앞으로 내가 담당할 설계 및 최적화 작업에 큰 도움이 될 것이다. 또한, 데이터 기반 의사결정이 소방 설비의 안전성과 효율성을 어떻게 향상시킬 수 있는지 직접 경험하며, 내가 추구하는 진로의 중요성과 가능성을 더욱 실감할 수 있었다. 이러한 경험은 나의 기술적 역량뿐만 아니라 문제를 바라보는 관점까지 넓혀 주었으며, 앞으로의 성장에 중요한 밑거름이 될 것이다.